TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics



LABORATORY WORK NO. 3

Polyalphabetic Ciphers: Playfair Cipher for the Romanian Language

Course: Cryptography and Information Security

Author:
Vitalie Vasilean
Student of group: FAF-241

Verified by:
Lecturer / Supervisor
Maia Zaica

Chișinău — 2026

Contents

1 Introduction and Laboratory Objectives **2**

2 Theoretical Foundations **2**
2.1 Monoalphabetic vs. Polygraphic Substitution 2
2.2 Romanian Alphabet Integration and Matrix Geometry 3
2.3 Matrix Construction Algorithm 3
2.4 Digraph Preparation Rules 4
2.5 Geometric Substitution Rules 4
2.5.1 Encryption Rules 4
2.5.2 Decryption Rules 4
2.6 Key Space Complexity 5

3 Architecture of the .NET Solution **5**
3.1 Playfair Matrix Model 5
3.2 Digraph Preparation and Encryption Engine 7

4 Implementation Details and Algorithmic Analysis **8**
4.1 Handling Diacritics and Unicode Compatibility 8
4.2 Circular Shift Arithmetic and Negative Modulo Prevention 9
4.3 Non-Overlapping Digram Splitting Mechanics 9

5 Experimental Results and Verification **9**
5.1 Test Case 1: Standard Verification with Double Letters 9
5.2 Test Case 2: Validation and Error Handling 10

6 Source Code and Repository **11**
6.1 Build and Execution Instructions 11

7 Conclusion **12**

1 Introduction and Laboratory Objectives

Monoalphabetic substitution ciphers suffer from an inherent cryptographic flaw: they preserve the underlying unigram frequency distribution of the natural language alphabet. An adversary analyzing a sufficiently long ciphertext can effortlessly break the cipher by mapping frequent ciphertext letters directly to frequent plaintext letters.

To counteract statistical frequency cryptanalysis, historical cryptologists introduced **polyalphabetic substitution ciphers** and **polygraphic substitution ciphers**. Polygraphic ciphers, such as the Playfair cipher invented by Sir Charles Wheatstone in 1854 and popularized by Lord Lyon Playfair, replace groups of letters (digraphs) simultaneously rather than substituting single symbols individually. By operating across pairs of characters, the Playfair cipher flattens the frequency distribution: instead of analyzing 26 or 31 individual letter frequencies, an attacker is confronted with $26^2 = 676$ (or $30^2 = 900$) distinct digraphic combinations.

The primary objectives of this laboratory work (specifically addressing **Task 3.1**) are:

1. **Study polyalphabetic and digraphic substitution principles**, analyzing how pair-wise substitution obscures statistical traits and increases cryptographic resilience.
2. **Adapt the Playfair cipher to the Romanian language**, which features 31 standard characters ($A - Z$ and 5 diacritics: Ă, Â, Î, Ș, Ț).
3. **Formulate and construct a** 6×5 **cryptographic matrix**, resolving the alphabet dimension mismatch by merging $J \to I$, yielding exactly 30 cells.
4. **Implement a robust C# (.NET 10.0) software suite** adhering to clean architecture principles (`Models`, `Services`, and `UI`), featuring:
   - Key validation enforcing a minimum length of 7 characters ($|K| \geq 7$).
   - Character set validation limiting inputs to the Romanian alphabet ($A-Z, a-z$, Ă, Â, Î, Ș, Ț) and providing descriptive user feedback.
   - Automatic cedilla normalization (ş/ţ $\to$ ș/ț) and case normalization.
   - Digraph splitting logic with automatic filler insertion (X, and Z when colliding with X) for identical adjacent characters and lonely terminal characters.
   - Forward (encryption) and backward (decryption) cyclic shift transformations across rows, columns, and rectangles.
5. **Verify cryptographic correctness** through rigorous empirical test runs, demonstrating round-trip consistency between plaintext, ciphertext, and deciphered output.

2 Theoretical Foundations

2.1 Monoalphabetic vs. Polygraphic Substitution

In a classical monoalphabetic substitution over alphabet Σ, a static bijective permutation $\pi : \Sigma \to \Sigma$ is applied character by character:

$$c_i = \pi(m_i), \quad \forall i \in \{0, 1, \ldots, L-1\} \tag{1}$$

Because this mapping is memoryless and position-independent, the probability distribution is completely invariant:

$$\mathbb{P}(C = \pi(x)) = \mathbb{P}(M = x) \tag{2}$$

In contrast, the **Playfair cipher** operates as a digraphic substitution cipher over Σ^2. Plaintext m is partitioned into consecutive pairs of characters:

$$m = (m_0, m_1), (m_2, m_3), \ldots, (m_{2k}, m_{2k+1}) \tag{3}$$

Each digram (m_{2i}, m_{2i+1}) is transformed into a ciphertext digram (c_{2i}, c_{2i+1}) according to geometric coordinates within an encryption grid $\mathcal{M}$. Because a given letter is encrypted differently depending on its partner in the digram, the single-letter frequency distribution is substantially flattened, rendering simple frequency analysis ineffective.

2.2 Romanian Alphabet Integration and Matrix Geometry

The standard Romanian alphabet consists of $N = 31$ letters:

$$\Sigma_{\text{RO}} = \{\text{A, Ă, Â, B, C, D, E, F, G, H, I, Î, J, K, L, M,}\\ \text{N, O, P, Q, R, S, Ș, T, Ț, U, V, W, X, Y, Z}\}$$

In the traditional English Playfair cipher, the 26 letters of the Latin alphabet are arranged in a $5 \times 5 = 25$ grid by merging I and J. For the Romanian language:

- A matrix of size 6×5 (or 5×6) yields $6 \times 5 = 30$ cells.
- Following the established Playfair convention (as detailed in the laboratory guideline), the letter **J** is replaced by **I** in the plaintext, reducing the alphabet to exactly 30 unique symbols:
$$|\Sigma_{\text{RO}} \setminus \{J\}| = 31 - 1 = 30 \tag{4}$$
- These 30 letters fit into a 6×5 matrix without empty cells or extraneous padding symbols.

2.3 Matrix Construction Algorithm

Let $K = (k_0, k_1, \ldots, k_{p-1})$ denote the secret key phrase. The encryption matrix $\mathcal{M} \in \Sigma^{6\times5}$ is constructed as follows:

1. Normalize K: convert all characters to uppercase, replace all occurrences of J with I, and normalize cedillas (ş → ș, ţ → ț).
2. Extract unique characters from K in order of their first appearance, discarding duplicate occurrences:
$$K_{\text{dedup}} = \text{Unique}(K) \tag{5}$$
3. Append all remaining letters of $\Sigma_{\text{RO}} \setminus \{J\}$ in standard alphabetical order that were not present in K_{dedup}.
4. Populate $\mathcal{M}$ row by row from top-left $(0, 0)$ to bottom-right $(5, 4)$:
$$\mathcal{M}[i, j] = \text{Alphabet}_{\text{combined}}[i \times 5 + j], \quad i \in \{0, \ldots, 5\},\ j \in \{0, \ldots, 4\} \tag{6}$$

2.4 Digraph Preparation Rules

Plaintext messages cannot be fed directly into the geometric substitution stage without preprocessing:

1. **Preprocessing:** Spaces and punctuation are stripped; all characters are uppercased; J is mapped to I.
2. **Duplicate Avoidance within a Pair:** If two identical characters occur within the same digram ($m_i = m_{i+1}$), a filler character (conventionally **X**) is inserted between them. If the character itself is X, the alternate filler **Z** is used:
$$(m_i, m_i) \longrightarrow (m_i, X), \quad \text{and } m_{i+1} \text{ begins the next pair} \tag{7}$$
3. **Odd-Length Padding:** If the resulting sequence has an odd number of characters after resolving duplicates, a trailing filler (**X**, or **Z** if terminal is X) is appended to complete the final digram.

2.5 Geometric Substitution Rules

Let (c_1, c_2) denote the output of encrypting digram (m_1, m_2). Let the coordinates in $\mathcal{M}$ be:
$$\text{Pos}(m_1) = (r_1, col_1), \quad \text{Pos}(m_2) = (r_2, col_2) \tag{8}$$

2.5.1 Encryption Rules

- **Rule 1 (Same Row, $r_1 = r_2$):** Shift cyclically to the right by 1 column:
$$c_1 = \mathcal{M}[r_1, (col_1 + 1) \bmod 5], \quad c_2 = \mathcal{M}[r_2, (col_2 + 1) \bmod 5] \tag{9}$$
- **Rule 2 (Same Column, $col_1 = col_2$):** Shift cyclically downward by 1 row:
$$c_1 = \mathcal{M}[(r_1 + 1) \bmod 6, col_1], \quad c_2 = \mathcal{M}[(r_2 + 1) \bmod 6, col_2] \tag{10}$$
- **Rule 3 (Rectangle, $r_1 \neq r_2 \wedge col_1 \neq col_2$):** Form the bounding rectangle and swap columns (retain row, take counterpart's column):
$$c_1 = \mathcal{M}[r_1, col_2], \quad c_2 = \mathcal{M}[r_2, col_1] \tag{11}$$

2.5.2 Decryption Rules

Decryption applies the exact inverse geometric transformations:

- **Same Row ($r_1 = r_2$):** Shift cyclically to the left by 1 column:
$$m_1 = \mathcal{M}[r_1, (col_1 - 1 + 5) \bmod 5], \quad m_2 = \mathcal{M}[r_2, (col_2 - 1 + 5) \bmod 5] \tag{12}$$
- **Same Column ($col_1 = col_2$):** Shift cyclically upward by 1 row:
$$m_1 = \mathcal{M}[(r_1 - 1 + 6) \bmod 6, col_1], \quad m_2 = \mathcal{M}[(r_2 - 1 + 6) \bmod 6, col_2] \tag{13}$$
- **Rectangle ($r_1 \neq r_2 \wedge col_1 \neq col_2$):** Identical to encryption:
$$m_1 = \mathcal{M}[r_1, col_2], \quad m_2 = \mathcal{M}[r_2, col_1] \tag{14}$$

2.6 Key Space Complexity

The total theoretical key space of a 6×5 Playfair matrix corresponds to all possible permutations of 30 distinct symbols:

$$|\mathcal{K}| = 30! \approx 2.6525 \times 10^{32} \tag{15}$$

Even after accounting for cyclic row and column shift equivalences and matrix transpositions (which reduce the number of cryptographically distinct grids by a factor of $6 \times 5 \times 2 = 60$), the effective key space remains approximately:

$$|\mathcal{K}_{\text{eff}}| = \frac{30!}{60} \approx 4.42 \times 10^{30} \tag{16}$$

This renders exhaustive brute-force searching entirely impossible by manual or computerized brute-force means.

3 Architecture of the .NET Solution

The application was engineered in **C# (.NET 10.0)** following modern object-oriented software engineering principles. The solution is structured into distinct architectural layers: `Models`, `Services`, and `UI`.

3.1 Playfair Matrix Model

Listing 1 displays the constructor of `PlayfairMatrix.cs`, illustrating how the key is processed and mapped to coordinates.

```
public class PlayfairMatrix
{
    public const int Row = 6;
    public const int Col = 5;
    char[,] matrix = new char[Row, Col];
    private readonly Dictionary<char, (int Row, int Col)>
   lookupDict = [];

    public PlayfairMatrix(string key)
    {
        StringBuilder result = new();
        HashSet<char> seen = [];

        // 1. Insert unique characters from normalized key
        foreach (char c in key)
        {
            if (!seen.Contains(c))
            {
                seen.Add(c);
                result.Append(c);
            }
        }

```

Architectural Decomposition of RomanianPlayfairCipher

- **Models Layer** (`PlayfairMatrix.cs`):
  - Encapsulates the 6×5 two-dimensional grid `char[6, 5]`.
  - Maintains an auxiliary coordinate dictionary `Dictionary<char, (int Row, int Col)>` providing $\mathcal{O}(1)$ position lookup during transformation.
  - Handles key deduplication and alphabet completion.
  - Exposes formatted grid visualization via `DisplayMatrix()`.
- **Services Layer**:
  - `AlphabetConfig.cs`: Acts as the single source of truth for the Romanian alphabet, diacritic normalization (ş → ș, ţ → ț), $J \to I$ replacement, and validation predicates (`IsValidKey`, `IsValidMessage`).
  - `PlayfairCipher.cs`: Implements stateless core cryptographic operations: non-overlapping digram preparation (`PrepareText`), forward encryption (`Encrypt`), and inverse decryption (`Decrypt`).
- **UI Layer** (`UserMenu.cs`):
  - Console interactive menu coordinating encryption and decryption workflows.
  - Input reading loops with immediate validation feedback for key length (≥ 7) and allowed characters.
  - Formatted output display.
- **Application Bootstrap** (`Program.cs`):
  - Initializes console I/O encodings to `System.Text.Encoding.UTF8` to guarantee proper terminal rendering of Romanian diacritics.

Figure 1: Modular Architecture of the Romanian Playfair Cipher System

```
23          // 2. Append remaining Romanian alphabet characters (30
    chars, no J)
24          foreach (char c in AlphabetConfig.RomanianAlphabet)
25          {
26              if (!seen.Contains(c))
27              {
28                  seen.Add(c);
29                  result.Append(c);
30              }
31          }
32
33          // 3. Populate 2D grid and lookup dictionary
34          int currentChrIndx = 0;
35          for (int i = 0; i < Row; i++)
36          {
37              for (int j = 0; j < Col; j++)
38              {
```

```
                char ch = result[currentChrIndx++];
                matrix[i, j] = ch;
                lookupDict[ch] = (i, j);
            }
        }
    }
}
```

Listing 1: PlayfairMatrix Constructor and Mapping Logic

3.2 Digraph Preparation and Encryption Engine

Listing 2 showcases the non-overlapping pair preparation loop and the geometric encryption algorithm in `PlayfairCipher.cs`.

```
public static List<(char, char)> PrepareText(string cleanText)
{
    int i = 0;
    List<(char, char)> pairs = [];
    while (i < cleanText.Length)
    {
        char first = cleanText[i];
        if (i == cleanText.Length - 1) // Lonely terminal character
        {
            pairs.Add((first, first == 'X' ? 'Z' : 'X'));
            i++;
        }
        else if (first == cleanText[i + 1]) // Double identical characters in pair
        {
            pairs.Add((first, first == 'X' ? 'Z' : 'X'));
            i++; // Advance by 1: second character starts the next pair
        }
        else // Distinct character pair
        {
            pairs.Add((first, cleanText[i + 1]));
            i += 2; // Advance by 2: both characters consumed
        }
    }
    return pairs;
}

public static string Encrypt(string plainText, PlayfairMatrix matrix)
{
    List<(char, char)> preparedText = PrepareText(plainText);
    StringBuilder cipheredChars = new();

    foreach ((char c1, char c2) in preparedText)
    {
```

```
34          var (r1, col1) = matrix.GetPosition(c1);
35          var (r2, col2) = matrix.GetPosition(c2);
36
37          char newLetter1, newLetter2;
38          if (r1 == r2) // Same row: shift right
39          {
40              newLetter1 = matrix.GetChar(r1, (col1 + 1) %
    PlayfairMatrix.Col);
41              newLetter2 = matrix.GetChar(r2, (col2 + 1) %
    PlayfairMatrix.Col);
42          }
43          else if (col1 == col2) // Same col: shift down
44          {
45              newLetter1 = matrix.GetChar((r1 + 1) %
    PlayfairMatrix.Row, col1);
46              newLetter2 = matrix.GetChar((r2 + 1) %
    PlayfairMatrix.Row, col2);
47          }
48          else // Rectangle: swap columns
49          {
50              newLetter1 = matrix.GetChar(r1, col2);
51              newLetter2 = matrix.GetChar(r2, col1);
52          }
53          cipheredChars.Append(newLetter1).Append(newLetter2);
54      }
55      return cipheredChars.ToString();
56  }
```

Listing 2: Text Preparation and Encryption in PlayfairCipher.cs

4 Implementation Details and Algorithmic Analysis

4.1 Handling Diacritics and Unicode Compatibility

A key technical challenge when targeting the Romanian alphabet in Windows console applications is encoding distortion. By default, legacy Windows Command Prompt and PowerShell hosts operate under code page 850 or 1252, causing Romanian characters such as Ă, Î, Ș, and Ț to render as replacement markers (?).

To resolve this issue, `Program.cs` explicitly configures console input and output encodings at application start:

```
Console.InputEncoding = System.Text.Encoding.UTF8;
Console.OutputEncoding = System.Text.Encoding.UTF8;
```

Furthermore, `AlphabetConfig.Normalize()` automatically standardizes legacy Romanian cedilla characters (Ş, Ţ) into the Unicode standard comma-below diacritics (Ș, Ț) specified by ISO/IEC 8859-16 and modern typography.

4.2 Circular Shift Arithmetic and Negative Modulo Prevention

In C#, the `%` operator computes the remainder of integer division rather than the true Euclidean modulo. For a negative dividend, `-1 % 5` evaluates to `-1` rather than `4`. When performing leftward (row) and upward (column) cyclic shifts during decryption, a naive decrement would produce negative array indices and cause runtime `IndexOutOfRangeException` crashes.

To ensure positive Euclidean indexing without branching statements, the dimension offset is added before modulo evaluation:

$$\text{col}_{\text{new}} = (col - 1 + \text{Cols}) \bmod \text{Cols} \tag{17}$$

$$\text{row}_{\text{new}} = (row - 1 + \text{Rows}) \bmod \text{Rows} \tag{18}$$

This guarantees that wrap-around from column 0 to column 4 and row 0 to row 5 operates smoothly.

4.3 Non-Overlapping Digram Splitting Mechanics

A crucial insight in the implementation of `PrepareText()` is index advancement. If two adjacent characters match (e.g., `LL` in `HELLO`), inserting an X resolves the collision. Crucially, the pointer must only increment by 1 ($i \leftarrow i + 1$). The second letter L is not discarded; rather, it becomes the first letter of the subsequent pair. Conversely, when two distinct letters are paired, both characters have been consumed, requiring an increment of 2 ($i \leftarrow i + 2$).

5 Experimental Results and Verification

To thoroughly validate the cryptographic correctness and operational reliability of the software, multiple test cases were executed.

5.1 Test Case 1: Standard Verification with Double Letters

- **Plaintext:** `"Hello World"`
- **Secret Key:** `"abcdefgh"` (length $8 \geq 7$)
- **Normalized Message:** `HELLOWORLD`
- **Prepared Digraphs:** `(H, E)`, `(L, X)`, `(L, O)`, `(W, O)`, `(R, L)`, `(D, X)`
- **Generated Matrix** (6×5)**:**

$$\mathcal{M} = \begin{bmatrix} \text{A} & \text{B} & \text{C} & \text{D} & \text{E} \\ \text{F} & \text{G} & \text{H} & \text{Ă} & \text{Â} \\ \text{I} & \text{Î} & \text{K} & \text{L} & \text{M} \\ \text{N} & \text{O} & \text{P} & \text{Q} & \text{R} \\ \text{S} & \text{Ș} & \text{T} & \text{Ț} & \text{U} \\ \text{V} & \text{W} & \text{X} & \text{Y} & \text{Z} \end{bmatrix}$$

The actual terminal execution session is captured below:

Console Terminal Session

```
Romanian Playfair Cipher
Choose an option:
1. Encrypt
2. Decrypt
3. Exit
1
Enter the message:
Hello World
Enter key (min 7 chars): abcdefgh
[      A      B      C      D      E  ]
[      F      G      H      Ă      Â  ]
[      I      Î      K      L      M  ]
[      N      O      P      Q      R  ]
[      S      Ş      T      Ţ      U  ]
[      V      W      X      Y      Z  ]
Encrypted Message: ÂCKYÎQBŞQMCY

Romanian Playfair Cipher
Choose an option:
1. Encrypt
2. Decrypt
3. Exit
2
Enter ciphertext (even length): ÂCKYÎQBŞQMCY
Enter key (min 7 chars): abcdefgh
[      A      B      C      D      E  ]
[      F      G      H      Ă      Â  ]
[      I      Î      K      L      M  ]
[      N      O      P      Q      R  ]
[      S      Ş      T      Ţ      U  ]
[      V      W      X      Y      Z  ]
Decrypted Message: HELXLOWORLDX
```

Analysis: The deciphered text `HELXLOWORLDX` perfectly matches the original message structure. The filler character `X` between `L` and `L` and the terminal padding `X` after `D` were cleanly recovered, demonstrating bidirectional mathematical consistency.

5.2 Test Case 2: Validation and Error Handling

The laboratory specification requires:

1. Rejecting keys shorter than 7 characters ($|K| < 7$).
2. Rejecting non-Romanian characters (digits, symbols) and suggesting the valid range (A-Z, Ă, Â, Î, Ș, Ț).
3. Rejecting odd-length ciphertexts during decryption.

The terminal log below demonstrates the defensive validation loop:

Console Terminal Session

```
Romanian Playfair Cipher
Choose an option:
1. Encrypt
2. Decrypt
3. Exit
1
Enter the message:
Test123
Char '1' is not allowd, only (A-Z, Ă, Â, Î, Ş, Ţ).
Enter the message:
TEST
Enter key (min 7 chars): short
Key 'SHORT' is too short (5 chars). Min length is 7.
Enter key (min 7 chars): CRIPTOGRAFIE
[      C      R      I      P      T  ]
[      O      G      A      F      E  ]
[      Ă      Â      B      D      H  ]
[      Î      K      L      M      N  ]
[      Q      S      Ş      Ţ      U  ]
[      V      W      X      Y      Z  ]
Encrypted Message: EHUR
```

As demonstrated, improper user inputs are intercepted gracefully without application termination, guiding the user until compliant inputs are provided.

6 Source Code and Repository

The complete source code and project configurations are hosted in the GitHub repository:

https://github.com/Vitalieww/CryptographyAndSecurityCourse/tree/main/lab_3

6.1 Build and Execution Instructions

To build and launch the application:

```
# Clone the repository
git clone
   https://github.com/Vitalieww/CryptographyAndSecurityCourse.git

# Navigate to the Romanian Playfair Cipher project
cd CryptographyAndSecurityCourse/lab_3/RomanianPlayfairCipher

# Build the project using .NET CLI
dotnet build
```

```
# Launch the interactive console suite
dotnet run
```

7 Conclusion

Through this laboratory work, the design, adaptation, and algorithmic mechanics of the **Playfair Cipher** for the Romanian language were systematically investigated and successfully implemented:

1. **Alphabet Adaptation and Dimension Reconciliation:** The 31-character Romanian alphabet was mapped to a compact 6×5 matrix by replacing J with I. This preserved all linguistic diacritics (Ă, Â, Î, Ș, Ț) while ensuring exact grid occupancy without redundant filler cells.

2. **Digraphic Substitution Resilience:** By substituting pairs of characters rather than single symbols, the Playfair cipher flattens the frequency distribution. Natural language letter patterns are masked, preventing trivial single-letter frequency attacks.

3. **Defensive Software Design:** The implemented C# solution guarantees data integrity through diacritic normalization, input validation loops, and robust modular arithmetic that eliminates negative index exceptions.

4. **Cryptographic Scope and Limitations:** While Playfair constitutes a marked security improvement over classical monoalphabetic substitution ciphers, it remains vulnerable to digraph frequency analysis and hill-climbing heuristics when large volumes of ciphertext are available. It serves as an intuitive pedagogical bridge connecting classical substitution to modern block cipher principles.

References

[1] Charles Wheatstone, *The Playfair Cipher System*, British Foreign Office Correspondence, 1854.

[2] David Kahn, *The Codebreakers: The Story of Secret Writing*, Scribner, 1996.

[3] William Stallings, *Cryptography and Network Security: Principles and Practice*, 7th Edition, Pearson, 2017.